

Clase 112 — Web cache poisoning y HTTP request smuggling

Parte: 4 — Seguridad de aplicaciones web · Fuente: PortSwigger Research (James Kettle) ⌚ Duración estimada: 130 min · Nivel: Experto

Objetivo

Explotar dos ataques avanzados a nivel de protocolo e infraestructura: el **web cache poisoning**, que envenena respuestas cacheadas para afectar a muchos usuarios, y el **HTTP request smuggling**, que abusa de discrepancias entre servidores frontend y backend al interpretar los límites de una petición. Son técnicas de alto nivel, muy premiadas en bug bounty.

⚠️ **Ética:** solo en labs propios/autorizados (PortSwigger). Estos ataques afectan a infraestructura compartida; nunca los pruebas en producción ajena.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** cómo las caches web y las cadenas de proxies procesan peticiones.
- 2. **Envenenar** una cache mediante cabeceras no incluidas en la clave.
- 3. **Distinguir** desincronización CL.TE, TE.CL y TE.TE.
- 4. **Explotar** request smuggling para envenenar respuestas y secuestrar peticiones.
- 5. **Recomendar** normalización y HTTP/2 end-to-end como defensa.

Temas

#	Tema	Por qué importa
1	Caches web y cache keys	Base del poisoning
2	Unkeyed inputs	Vector del envenenamiento
3	Cadena frontend/backend	Base del smuggling
4	CL.TE, TE.CL, TE.TE	Tipos de desincronización
5	Impacto: hijack y bypass	Traducir a daño real
6	HTTP/2 downgrade	Superficie moderna
7	Defensa: normalizar, rechazar ambiguo	Cierre del fallo

Definiciones y características

- **Web cache poisoning:** inyectar contenido en una respuesta cacheada que se sirve a otros usuarios. Característica: amplifica un input a muchas víctimas.
- **Cache key:** conjunto de campos que identifican una respuesta en cache. Característica: los inputs no incluidos (unkeyed) son el vector.
- **HTTP request smuggling:** aprovechar que frontend y backend delimitan las peticiones de forma distinta. Característica: "contrabandea" una petición dentro de otra.
- **CL.TE / TE.CL:** discrepancia entre `Content-Length` y `Transfer-Encoding`. Característica: cada extremo usa una cabecera distinta.
- **TE.TE:** ambos usan `Transfer-Encoding` pero uno se ofusca. Característica: se evade con obfuscación del header.
- **Response queue poisoning:** desalinear respuestas para servir la de otro usuario. Característica: impacto grave del smuggling.

Herramientas y preparación

- **PortSwigger labs** de request smuggling y cache poisoning.
- **Burp** con la extensión **HTTP Request Smuggler** y **Param Miner**.
- Comprensión de HTTP/1.1 y HTTP/2.

Laboratorio guiado

 Solo en labs propios/autorizados.

1. **Cache poisoning:** usa Param Miner para descubrir cabeceras no incluidas en la clave (unkeyed).
2. Inyecta un valor malicioso en esa cabecera y comprueba si se refleja y se **cachea**.
3. Verifica el envenenamiento accediendo a la URL sin la cabecera (recibes el contenido inyectado).
4. **Request smuggling:** con HTTP Request Smuggler, detecta la desincronización (CL.TE/TE.CL).
5. Construye manualmente una petición con `Content-Length` y `Transfer-Encoding` conflictivos para "contrabandear" una segunda petición.
6. Demuestra el impacto: captura la petición de otro usuario o envenena la cola de respuestas en el lab.
7. Documenta el tipo de desincronización, el payload y el impacto.

Ejercicios

1. Explica con un diagrama cómo la cache key deja fuera ciertos inputs.
2. Envenena una cache vía cabecera unkeyed en un lab.
3. Diferencia CL.TE, TE.CL y TE.TE con ejemplos de peticiones.
4. Construye a mano una petición de smuggling CL.TE.
5. Explica cómo HTTP/2 end-to-end mitiga el smuggling clásico.
6. Propón defensas: normalización en el frontend y rechazo de peticiones ambiguas.

Reto verificable

Resuelve un lab de **HTTP request smuggling** de PortSwigger (CL.TE o TE.CL) y un lab de **cache poisoning**, demostrando impacto en ambos. **Criterio de aceptación:** ambos labs quedan resueltos, entregas las peticiones exactas (cabeceras conflictivas / input unkeyed), la evidencia del impacto y la defensa correspondiente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El input no se cachea	Está en la clave o hay <code>Cache-Control: no-store</code> ; busca otro header
Smuggling no desincroniza	Ambos usan la misma cabecera; prueba TE.TE con obfuscación
El lab usa HTTP/2	El vector clásico cambia; usa técnicas de downgrade/H2
Respuestas inconsistentes	Timing sensible; repite y ajusta
Param Miner sin hallazgos	Amplía el diccionario de cabeceras

Preguntas frecuentes

 **¿Por qué el smuggling es tan potente?** Porque puede secuestrar peticiones de otros usuarios, saltar controles del frontend y envenenar respuestas a escala.

 **¿HTTP/2 elimina el smuggling?** El clásico de HTTP/1.1 sí se mitiga con HTTP/2 end-to-end, pero aparecen variantes (H2.CL, downgrade) si hay traducción a HTTP/1.1.

 **¿Qué hace peligroso al cache poisoning?** Que un solo input malicioso, una vez cacheado, se sirve a todos los usuarios que piden esa URL.

Referencias

- PortSwigger — HTTP request smuggling: <https://portswigger.net/web-security/request-smuggling>
- PortSwigger — Web cache poisoning: <https://portswigger.net/web-security/web-cache-poisoning>
- James Kettle, investigaciones de PortSwigger Research.
- RFC 7230 (HTTP/1.1 message syntax).

Siguiendo clase

Clase 113 - Ataques del lado del cliente: CORS, postMessage y prototype pollution